

SD TCL - The Command Processor

How a command line is read, what the VOC does with it, and how sentences and paragraphs run.

TCL is the prompt you get when you start SD. It is not a shell: it does not search a path, it does not expand wildcards, and it has no built-in commands of its own. **Every command you type is a record in a file**, and this page is mostly about that file and about the small set of verbs that control the session you are typing into.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case, which is what this port uses on disk. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

The prompt

The command processor is a BASIC program, `$CPROC`, and it runs a loop: show any waiting messages, print the prompt, read a line, dispatch it, repeat. The prompt character is `?` and it is what `system(26)` reports.

Messages sent by `message` are printed at the prompt, not when they arrive. They queue in the `$ipc` file and the loop drains that queue immediately before each prompt, so a message sent to a session sitting at TCL appears at once and a message sent to a session running a long report appears when the report finishes.

`report.src` turns on a line after every command reading `SRC =` and the value of `@system.return.code`. It takes `on`, `off`, or nothing at all, in which case it toggles.

Where a command goes

The first word of the line is looked up in the account's `voc` file. **What comes back decides everything else**, because field 1 of a VOC record begins with a type character:

V	a verb — the common case
K	a query-processor keyword, which may also be a verb (see below)
S	a sentence: one stored command line
PA	a paragraph: several stored command lines, with flow control
M	a menu

Q	an indirect pointer to a file in another account
F	a file
R	a pointer to a record in another file, re-read and re-parsed
X	miscellaneous data — not a command
D	a data descriptor, which is what dictionary records are

The rest of field 1 is free text and is ignored, so a verb record whose field 1 reads `Verb to compile SD BASIC program` is type `V` and the sentence is a comment. **This is worth knowing before you edit a VOC record by hand:** the first character is load-bearing and the remaining thirty are not.

NAMES ARE TRIED AS TYPED, THEN LOWER, THEN UPPER

This port shipped a change here. Where SD used to try the name exactly as typed and then in upper case, it now tries **as typed, then lower case, then upper case**. That is what lets a lower-case `voc`, `bp` and `newvoc` on disk coexist with code and habits that still type names in upper case, and it is why you can type `list voc` or `LIST VOC` and reach the same file.

Account names are the deliberate exception. They are still folded upward, and that is what makes signing in case-insensitive.

A HYPHEN IS ACCEPTED WHERE THE VERB HAS A DOT

After the three case attempts, the command processor tries the name again with any hyphens changed to dots. **So every dotted verb also answers to a hyphen.**

```
:clear-select
Cleared numbered select list 0
:clear.select
Cleared numbered select list 0
```

This is a spelling variant rather than a second verb, and two controls show it:

```
:zzz-nosuch
zzz-nosuch is not in your VOC
:ct voc create-account
Record 'create-account' not found
```

The first shows the fallback is not accepting anything at all. The second shows there is no `create-account` record in the VOC — the resolution happens in the command processor, not in the file.

The dotted spelling is the documented one and is what every page here uses. The hyphen form is supported, and it is worth knowing because a typed hyphen will not produce the error you expect.

IF THE VOC HAS NOTHING, THE CATALOGUE IS TRIED

A name that is not in the VOC is looked for in the **private catalogue** of the account and then in the **global catalogue**, `gcat`. A hit is treated exactly as though the VOC had held `V` / `CA` / *name*. This is why a program you have catalogued can be run by typing its name with no VOC record of its own.

If neither has it, the command is refused by name.

A KEYWORD CAN BE A VERB

Four records that ship are a keyword **and** a verb in one record: `break`, `count`, `display` and `off`. Their field 2 is the keyword number the query processor uses, and **fields 3 onward are a complete verb record**, which the command processor re-parses when the name is typed as a command.

This matters for counting what an account has. A tally of VOC records whose field 1 begins with `V` misses all four, and `count` is not a marginal verb. A standard account has **82** verbs, not 78.

What kind of thing a verb is

Field 2 of a verb record says how to run it, and field 3 says what to run:

<code>CA</code>	<i>name</i>	a catalogued program — 97 of the shipped verbs
<code>IN</code>	<i>n</i>	internal verb <i>n</i> , handled by the command processor itself — 45
<code>OS</code>	<i>text</i>	an operating-system command — <code>sh</code> and <code>!</code>
<code>CS</code>	<i>path</i>	a locally catalogued function

Field 4 carries dispatch options and **field 5 names a security subroutine**. If field 5 is present, that subroutine is called before the verb runs and can refuse it, in which case you are told the command is restricted. None of the shipped verbs uses field 5 — **the tiering in this port is done by giving or withholding the VOC record**, not by a security subroutine — but the mechanism is there for a site that wants a verb guarded rather than absent.

There are **41 internal verbs**, numbered, and the numbers are positional in the command processor's dispatch list. Several names share one: `off` and `quit` are both internal verb 1, `clr` and `cs` are both 2, and each of `clear.data`, `clear.input`, `clear.prompts` and `clear.select` has a run-together spelling (`cleardata` and the rest) pointing at the same number.

Sentences and paragraphs

A **sentence** is a VOC record holding one command line. Typing its name runs that line with anything else you typed appended, so a sentence is a command with its first arguments filled in.

A **paragraph** holds several lines and is the closest thing SD Core has to a shell script. Six verbs exist for paragraphs and **four of them are refused outside one**:

<code>if</code> <i>value op value ; command</i>	conditional. Only inside a paragraph
<code>go</code> <i>label</i>	jump to a label. Only inside a paragraph
<code>stop</code>	end this paragraph, return to the caller
<code>abort</code> <i>{text}</i>	end the paragraph and everything that called it
<code>display</code> <i>text</i>	write a line to the terminal
<code>pause</code>	print a <i>press return</i> prompt and wait

`go` searches forward from the current line for a line beginning with the label; **a label prefixed with @ is searched for from line 1 instead**, which is how you jump backwards. Labels carry a trailing colon, and one is added for you if you leave it off.

`display` understands a leading cursor-position clause, so `display @(0,0) ready` positions and then writes. A trailing colon suppresses the newline.

`data` **lines are read ahead of the verb, not executed in sequence.** Before a verb runs, the command processor scans forward over any immediately following `data`, comment and blank lines and stacks their text as typed-ahead input. So the `data` lines that feed a program are written *after* the command that runs it, and they must follow it without an intervening ordinary command.

Paragraphs nest, and `abort` unwinds all of them.

Continued in

SD TCL - The Command Stack — the command stack, the @variables, and the session verbs.